

Specula (Specula-Agent)

Autonomous Multi-Agent Digital Forensic & Incident Response (DFIR) Framework

Project Focus: Autonomous Multi-Agent DFIR Scene Reconstruction

Domain: AI & Cybersecurity (Digital Forensics)

Academic Context: SDC-II Research Project (2025–2026)

Department: Computer Science & Engineering

1. Project Explanation

1.1 Project Idea

Specula (Specula-Agent) is an autonomous multi-agent artificial intelligence framework engineered to reconstruct digital crime scenes from fragmented, heterogeneous evidence without requiring continuous manual analyst intervention. The system coordinates a team of thirteen specialized AI agents—each responsible for a distinct domain of forensic investigation—through a centralized Digital Forensic Knowledge Graph (DFKG) that serves as shared investigative memory.

Rather than feeding raw logs directly into a single language model—which fails due to context window limits, token budget exhaustion, and hallucination—Specula distributes evidence analysis across parallel specialist agents, compresses evidence intelligently using entropy-based semantic distillation, grounds all reasoning in verifiable graph citations, and produces court-ready forensic reports satisfying Daubert legal admissibility standards.

1.2 Problem Statement

Modern digital forensic investigations face four compounding crises that make manual investigation increasingly untenable:

Crisis Domain	Scale / Operational Impact	Forensic Consequence
Log Volume Explosion	A clean Windows Server installation generates over 4,000,000 events in a short timeframe.	Critical evidence is buried in noise; manual triage is impossible at speed.

Context Window Limits	Single LLMs cannot process millions of raw log records without exhausting token budgets.	AI-assisted investigation truncates evidence and loses investigative continuity.
Manual Correlation	Analysts must manually stitch fragmented alerts across dozens of log types and systems.	Process is slow, expensive, error-prone, and leads to severe investigator burnout.
Legal Inadmissibility	Unconstrained LLM outputs hallucinate IP addresses, file paths, and timelines.	AI-generated findings are rejected under Daubert legal standards in court proceedings.

1.3 Project Objectives

- **Reduce Average Investigation Duration:** Lower investigation duration from 45 minutes to 5 minutes (an 89% time reduction) via autonomous multi-agent parallelization.
- **Achieve High Investigative Accuracy:** Reach 95% evidence correlation accuracy and 96% attack timeline reconstruction accuracy.
- **Mitigate AI Hallucinations:** Reduce the AI hallucination rate to below 5% through adversarial debate and DFKG-grounded reasoning.
- **Ensure Legal Admissibility:** Produce forensic reports that satisfy Daubert legal admissibility standards via cryptographic Merkle provenance.
- **Multi-Source Ingestion:** Handle 14 heterogeneous input categories spanning OS logs, network PCAPs, memory dumps, and cloud audit trails.
- **Explainable Reasoning:** Provide auditable chain-of-thought reasoning traceable directly to raw evidence for every finding.

2. Base Paper & Literature Grounding

2.1 Primary Base Paper Selected

Selected Primary Base Paper: "GenDFIR: Advancing Cyber Incident Timeline Analysis Through Retrieval-Augmented Generation and Large Language Models" — Authored by Fatma Yasmine Loumachi, Mohamed Chahine Ghanem, and Mohamed Amine Ferrag (2024).

2.2 Methodology & Key Contributions of the Base Paper

The GenDFIR framework introduced an innovative approach to automating cyber incident timeline analysis by combining Llama 3.1 8B in a zero-shot setting with a Retrieval-Augmented Generation (RAG) agent. Its core methodology comprises:

- **Stage 1 — Data Preprocessing & Structuring:** Transforms raw event logs (CSV format) into structured textual document knowledge bases representing the incident.
- **Stage 2 — Context Retrieval & Semantic Enrichment:** Uses embedding models (mxbai-embed-large) and cosine similarity to retrieve relevant incident events (top-k evidence) based on user prompts.
- **Stage 3 — LLM Decoder Timeline Synthesis:** Processes weighted event contexts through Llama 3.1 8B to generate plain-text timeline reports, root cause analyses, and mitigation suggestions.

2.3 Supporting Literature Matrix

Paper / Source	Domain Focus	Key Contribution to Specula
GenDFIR (Loumachi et al., 2024)	Timeline Analysis & RAG	Establishes zero-shot RAG log retrieval, DFIR prompt engineering, and context enrichment baselines.
MADIK Framework (London Met Uni)	Multi-Agent Coordination	Establishes agent-to-agent coordination patterns across artifact-specific forensic specialists.
CFA-Bench & CyberSleuth	Network Forensics	Provides ground-truth evaluation datasets for network forensics with real PCAPs and CVE detection tasks.
DFIR-Metric Benchmark	Evaluation Methodology	Introduces Task Understanding Score (TUS) rewarding partial correctness in multi-step reasoning.
ForensicsData (ANY.RUN Triplets)	Malware Behavior	5,000+ Q-C-A triplets used to benchmark the Malware Behavior Agent narrative accuracy.

Analysis of Competing Hypotheses (ACH)	Adversarial Reasoning	Structured analytic technique adapted as the Evidentiary Adversarial Debate protocol for hallucination mitigation.
OCSF Standard	Log Normalization	Universal ingestion schema standardizing evidence across cloud, endpoint, and network telemetry.
Verifiable Conversation Transcripts (VCT)	Chain-of-Custody	Three-tier Merkle structure making AI reasoning chains tamper-evident under Daubert admissibility standards.

2.4 How Specula Extends and Improves Upon Existing Work

The primary base paper, **GenDFIR** (*Loumachi et al., 2024*), introduced a zero-shot framework combining a single Large Language Model (**Llama 3.1 8B**) with a standard Retrieval-Augmented Generation (**RAG**) pipeline. In GenDFIR, raw event logs are converted into flat textual document chunks, indexed via vector embeddings (mxbai-embed-large), and queried using cosine similarity to assemble an incident timeline summary.

While GenDFIR demonstrated the feasibility of using LLMs for DFIR timeline generation, it exhibits critical operational bottlenecks:

1. **Single-Agent Bottleneck:** Relies on a single RAG agent without domain-specialized workers.
2. **Flat Vector Retrieval Failure:** Cosine similarity over isolated log chunks misses multi-hop, non-linear lateral movement and entity relationships.
3. **Context Window Flooding:** Has no algorithmic log compression or entropy triage layer, leaving it vulnerable to context window exhaustion when processing millions of enterprise events.
4. **Unconstrained Hallucination Risk:** Generates summaries without cryptographic proof or mandatory citation checks, failing judicial evidence standards (**Daubert standard**).
5. **Narrow Analytical Scope:** Lacks active capabilities for anti-forensic timestamping detection, AI-code stylometry, zero-trust command execution, automated attack graph traversal, or dynamic malware detonation.

Complete Specula Feature Set & Improvements Over GenDFIR

1. Semantic Compression and Entropy-Based Log Distillation

- **Specula Specification:** Operates as a two-layer system. The *Forensic Preservation Layer* first commits all raw logs immutably to **Quickwit** and hashes them into the Verifiable Conversation Transcripts (**VCT**) chain using Go/Rust SIMD hashing at the edge. The *Analytical Abstraction Layer* then applies **Drain3** template extraction followed by **SimHash** similarity checking to prevent template poisoning attacks. It uses **MiniBatchKMeans** clustering with cosine distance to collapse low-entropy events into summaries while retaining high-entropy anomalous events intact. Reduces LLM input volume by 90%+ without legally compromising the original evidence corpus.
- **Specula Improvement over GenDFIR:** GenDFIR feeds raw text chunks directly into its vector database and prompt context, causing rapid token budget exhaustion when handling enterprise-scale logs (e.g., 4M+ Windows events). Specula introduces **information-entropy log distillation**, filtering out syntactic noise before LLM ingestion while preserving critical forensic indicators.

2. Digital Forensic Knowledge Graph (DFKG) and Topological GraphRAG

- **Specula Specification:** Replaces linear supertimelines with a **Neo4j** relational ontology where nodes represent forensic entities (users, devices, IPs, processes, files, malware, ATT&CK techniques) and edges represent observed actions between them. Deterministic UIDs generated by hashing source device ID, file path, database name, and row number prevent duplicate nodes across concurrent agents. GraphRAG queries combine **FAISS IndexIVFPQ** cosine similarity search with **APOC-bounded Breadth-First Search (BFS)** subgraph expansion (max 3 hops, max 100 nodes, max 300 relationships) to retrieve structured context for LLM reasoning. Supernode detection pre-checks node degree before BFS to prevent traversal explosions.
- **Specula Improvement over GenDFIR:** (*Feature not present in GenDFIR*) GenDFIR relies exclusively on flat vector similarity over text chunks, which loses relational context across systems. Specula transitions from flat text RAG to **Topological GraphRAG**, mapping multi-hop lateral movement and complex causal subgraphs across heterogeneous endpoints.

3. Algorithmic Temporal Normalization and Timestamping Detection

- **Specula Specification:** Detects anti-forensic timestamp manipulation by requiring three conditions simultaneously: the \$STANDARD_INFORMATION Created timestamp predates the volume creation date, the USN Journal shows a BasicInfoChange event with no corroborating application log entry within ± 5 seconds, and the timestamp precision shows suspicious microsecond rounding. Replaces static rules with an

XGBoost classifier trained on a SMOTE-balanced feature vector of timestamp deltas, precision bitmask fields, and USN reason codes. Time anchor correlation matrices using linear regression on simultaneous multi-device events resolve clock drift and normalize all timestamps to UTC baseline prior to DFKG ingestion.

- **Specula Improvement over GenDFIR:** (*Feature not present in GenDFIR*) GenDFIR assumes raw log timestamps are truthful and chronologically accurate. Specula actively identifies, isolates, and mathematically corrects **anti-forensic timestomping** and multi-device clock skew before timeline synthesis occurs.

4. Evidentiary Adversarial Debate and Analysis of Competing Hypotheses (ACH)

- **Specula Specification:** A three-agent **LangGraph** debate framework where the *Proponent Agent* constructs a hypothesis with mandatory DFKG node UID citations, the *Critic Agent* independently queries the DFKG for counter-evidence and argues alternative explanations (also with mandatory citations), and the *Judge Agent* evaluates arguments using an ACH evidence matrix. The Judge automatically rejects any argument not backed by explicit DFKG citations. Runs up to a configurable maximum of 3 debate rounds, terminating early when the confidence score delta between agents falls below 0.05. Softmax credibility weighting with Laplace smoothing handles the cold-start problem. LangGraph state stores only UID references, with full argument content held in **Redis** to prevent state bloat.
- **Specula Improvement over GenDFIR:** GenDFIR relies on single-pass, unvalidated LLM generation, which remains susceptible to parametric hallucinations and bias. Specula enforces a structured **Adversarial ACH Debate**, ensuring that any ungrounded assertion is flagged as `PARAMETRIC_KNOWLEDGE_REJECTED` and discarded.

5. Verifiable Conversation Transcripts (VCT) and Cryptographic Provenance

- **Specula Specification:** Secures the investigative chain of custody through a three-tier cryptographic structure:
 1. *Atomic Level:* Every prompt-response pair is SHA-256 hashed and linked into a hash chain.
 2. *Session Level:* Completed sub-task chain tails are aggregated into a Merkle tree using pymerkle with incremental leaf support.
 3. *Case Level:* All session roots aggregate into a final Merkle root signed by both the server RSA key and the human investigator's digital certificate, anchored to **Hyperledger Fabric** via fabric-sdk-py.

A **gRPC gossip protocol** with a 30-second timeout window and 2-peer confirmation requirement detects view forks from compromised agents without false alerts from legitimate network partitions. Satisfies Daubert admissibility standards.

- **Specula Improvement over GenDFIR:** *(Feature not present in GenDFIR)* GenDFIR provides no tamper-evident guarantees for its internal AI reasoning chain. Specula provides **cryptographic chain-of-custody tracking** for every LLM reasoning step, rendering the generated output legally defensible in court under **Daubert legal standards**.

6. LLM-Generated Code Stylometry and Authorship Attribution

- **Specula Specification:** Analyzes recovered malicious scripts to determine whether they were human-authored or generated by a specific LLM. A language detection step using linguist routes scripts to the appropriate parser—Python's built-in ast module, javalang for Java, pycparser for C, bashlex for shell scripts, and a subprocess-wrapped PowerShell AST parser for PowerShell. Extracts AST node type n-gram frequencies, Program Dependency Graph (PDG) structural features, identifier naming patterns, and control flow complexity metrics. An **XGBoost classifier** trained on a novel synthetic benchmark dataset (scripts generated by GPT-4, Claude, Gemini, and human authors) outputs a class label with an explicit confidence score, acknowledging inter-model boundaries as probabilistic rather than deterministic.
- **Specula Improvement over GenDFIR:** *(Feature not present in GenDFIR)* GenDFIR cannot analyze or attribute malicious code scripts. Specula introduces **AI Code Stylometry**, allowing investigators to determine if recovered malware scripts were generated by specific LLMs or human actors.

7. Zero-Trust Guardrail Execution

- **Specula Specification:** A mandatory three-tier validation layer that intercepts all agent outputs before execution:
 - *Tier 1 (<5ms):* Runs deterministic regex and AST signature matching after Unicode NFKC normalization and zero-width character stripping to catch homoglyph and encoding-based injection bypass attempts.
 - *Tier 2 (<50ms):* Runs a local fine-tuned **MiniLM vector classifier** for SAFE/UNSAFE semantic intent classification on write operations only.
 - *Tier 3 (1–3s):* Reserves full secondary LLM semantic validation exclusively for containment actions and high-risk DFKG writes.

Implements parameterized Cypher queries throughout to prevent graph injection attacks.

- **Specula Improvement over GenDFIR:** *(Feature not present in GenDFIR)* GenDFIR lacks security filters to protect itself against adversarial log inputs. Specula implements a **Zero-Trust Guardrail Architecture** to prevent indirect prompt injection (e.g., OWASP LLM05) embedded inside malicious log telemetry from hijacking agent execution.

8. Dedicated Fraud and Insider Threat Agent

- **Specula Specification:** Ingests raw message body text, DLP alerts, UEBA scores, and file access patterns from mandatory fields in email, messaging, and UEBA input sources. Runs **spaCy NER** on communications to identify mentions of sensitive systems, competitors, and behavioral indicators. Uses an **LSTM Autoencoder** trained on 90-day user activity baselines from the CERT dataset to score behavioral anomalies by reconstruction error. Applies **One-Class SVM** for users with fewer than 100 message samples, switching to **HDBSCAN** only when sufficient history exists. Uses ProsusAI/finbert or a fine-tuned CERT-corpus model for sentiment analysis to prevent domain mismatch.
- **Specula Improvement over GenDFIR:** *(Feature not present in GenDFIR)* GenDFIR focuses strictly on explicit system log events and cannot detect non-signature behavioral anomalies or corporate insider threats. Specula adds a **specialized Insider Threat Agent** capable of modeling semantic sentiment and behavioral baseline deviations across enterprise messaging and UEBA data streams.

9. Formalized Human-in-the-Loop (HITL) Approval Gate

- **Specula Specification:** A **FastAPI + React** dashboard that the Supervisor Agent escalates to when confidence scores fall below 0.7, adversarial debate produces unresolved conflict, blast radius exceeds the predefined threshold, or any active containment action is proposed. Presents the human analyst with the full ACH evidence matrix, blast-radius-ranked asset list, Explainability Agent chain-of-thought trace, and VCT session Merkle root for verification. Analyst actions: APPROVE, REJECT, or REQUEST_CLARIFICATION. Implements a tiered timeout policy: low-severity findings auto-approve after 4 hours, medium-severity escalate to a secondary analyst after 2 hours (with auto-approval after 8 hours), and high-severity actions never auto-approve (notifying SOC leadership via webhook after 1 hour). Analyst agreement rate is tracked as the primary evaluation metric.
- **Specula Improvement over GenDFIR:** *(Feature not present in GenDFIR)* GenDFIR operates as an unmonitored "black box" text generator without formal escalation points. Specula embeds a **legally mandated Human-in-the-Loop gate**, ensuring human oversight over critical findings and automated containment commands.

10. Dynamic Attack Graph Construction

- **Specula Specification:** Ingests vulnerability and asset inventory data (14th input source) from OpenVAS, Nessus, or Qualys APIs and cross-references with DFKG host nodes. Queries a locally cached **DuckDB** table of daily EPSS score dumps from FIRST.org rather than making live API calls to prevent rate limiting. Models the network as a directed networkx graph where edge weights use a negative log transformation:

$$W_{\text{transformed}} = -\ln(\text{CVSS}_{\text{base}} \times \text{EPSS} \times \gamma_{\text{precondition}} + \epsilon)$$

so Dijkstra's algorithm correctly identifies the most dangerous exploitation path rather than the safest one. The precondition confirmation factor $\gamma_{\text{precondition}} = 1.0$ when forensic evidence in the DFKG confirms the precondition, and 0.3 when unconfirmed. Unchained vulnerabilities with EPSS above 0.3 are flagged as residual risk and ranked for remediation output.

- **Specula Improvement over GenDFIR:** (*Feature not present in GenDFIR*) GenDFIR cannot assess vulnerability chains or post-incident residual risk. Specula dynamically constructs **forensically weighted attack graphs**, combining live forensic evidence with EPSS/CVSS scores to map viable attack paths and prioritize post-incident patching.

11. Malware Behavior Agent with Tiered Dynamic Analysis

- **Specula Specification:** Operates as a three-tier pipeline selecting analysis depth based on sample complexity and available infrastructure:
 - *Tier 1 (<1s):* Runs YARA rule matching via yara-python against the MalwareBazaar ruleset plus custom rules derived from previous DFKG malware nodes.
 - *Tier 2 (~5s):* Runs Speakeasy user-mode Windows emulation requiring no hypervisor—handles shellcode, DLLs, and standard executables.
 - *Tier 3:* Escalates to either CAPE Sandbox on bare-metal KVM hardware or an external API (ANY.RUN, Hybrid Analysis) when Speakeasy reports evasion or insufficient coverage.

All tiers output structured behavioral telemetry that the agent writes into the DFKG as typed edges (CALLED_API, DROPPED, CONNECTED_TO), enabling automated ATT&CK mapping. **DRAKVUF VMI** on Xen is supported as a gold-standard upper tier when hypervisor infrastructure is available, validated against versioned Pydantic schema models to catch parser-version mismatches.

- **Specula Improvement over GenDFIR:** (*Feature not present in GenDFIR*) GenDFIR has no capability to analyze or detonate malicious binaries. Specula integrates **tiered dynamic malware detonation and VMI telemetry ingestion**, mapping behavioral execution outputs directly into the shared DFKG memory.

12. Investigative Synthesis and Blast Radius Quantification

- **Specula Specification:** The final synthesis feature that runs after all specialist agents complete. Computes blast radius as a composite score of affected node count weighted by asset sensitivity classification, plus log-scaled estimated exfiltration volume derived from file size properties on confirmed exfiltration edges in the DFKG:

$$BR = (N_{\text{affected}} \times \bar{W}_{\text{sensitivity}}) + \ln(\text{Bytes}_{\text{exfiltrated}} + 1)$$

Performs threat actor attribution by computing both Jaccard set similarity and **Smith-Waterman sequence alignment scores** between the observed TTP sequence and MITRE ATT&CK group profiles, combining them as a weighted score ($0.4 \times \text{Jaccard} + 0.6 \times \text{Smith-Waterman}$) to reward matching TTP order rather than just TTP presence. Returns top-3 candidate threat actors with similarity scores and confidence bounds. Produces a ranked asset impact list fed directly to the HITL Approval Gate dashboard and outputs court-ready PDF reports via **ReportLab**.

- **Specula Improvement over GenDFIR:** GenDFIR outputs unstructured, plain-text narrative summaries without quantitative impact scoring or sequence-aligned attribution. Specula provides **mathematical blast radius quantification**, sequence-aligned TTP threat actor attribution, and automated court-ready PDF report synthesis.

Capability / Dimension	GenDFIR Baseline (Loumachi et al., 2024)	Specula Framework
System Architecture	Single LLM RAG Agent (Llama 3.1 8B)	13 Specialized Agents across 5 Cognitive Tiers
Context Retrieval	Flat Vector Search (Cosine Similarity over text chunks)	Topological GraphRAG (Neo4j DFKG + APOC BFS + FAISS IndexIVFPQ)
Log Scale Handling	Uncompressed Text Prompts (Token Bottleneck)	Entropy-Based Distillation (Drain3 + SimHash + MiniBatchKMeans, 90%+ Compression)
Hallucination Mitigation	Unconstrained Single-Pass Generation	Evidentiary ACH Debate (Proponent/Critic/Judge with mandatory DFKG citations)
Legal Admissibility	None (Unverified text output)	Cryptographic Provenance (3-Tier Merkle Trees + Hyperledger Fabric, Daubert Standard)
Timestomping Analysis	None (Assumes raw timestamps are accurate)	MFT SlvsFN Checks + XGBoost Classifier + Time Anchor Regression
Code Stylometry	None	AST / PDG Parsing across 6 Languages + XGBoost AI-Authorship Classifier
Security / Guardrails	None (Vulnerable to Indirect Prompt Injection)	3-Tier Zero-Trust Validation (Regex + MiniLM Classifier + Parameterized Cypher)
Insider Threat Analysis	None	LSTM Autoencoder + FinBERT Sentiment Analysis on corporate comms & UEBA
Human-in-the-Loop	None	Formalized HITL Gate (React Dashboard with Tiered Timeouts & Agreement Tracking)
Vulnerability	None	Dynamic Attack Graph (Dijkstra Traversal on negative

Analysis		$\log CVSS \times EPSS \times \gamma$
Malware Detonation	None	Tiered Sandbox Execution (YARA → Speakeasy → CAPE / DRAKVUF VMI)
Threat Attribution	Unstructured text mention	Sequence-Aligned Attribution ($0.4 \times \text{Jaccard} + 0.6 \times \text{Smith-Waterman}$)
Impact Scoring	Qualitative summary	Quantitative Blast Radius Formulation ($BR = N \cdot W + \ln(\text{Bytes} + 1)$)

2.5 Agent-Model Rationale Matrix

Agent	Model	Inference	Rationale/Justification
Supervisor	Nemotron-3 Ultra	Together AI via HF	Top-tier reasoning for orchestration, dead-end detection, and routing decisions
Evidence collection	Qwen2.5-7B-Instruct	Novita/Together via HF	Mostly tool-calling/fetching, low reasoning load — cheapest model that's reliable at function calling
Log analysis	Qwen2.5-72B-Instruct	Novita/Together via HF	Needs strong pattern recognition over high-volume normalized logs
Network forensics	Llama3.3-70B-Instruct	Groq via HF	Comparable capability tier to log analysis, keeps the two parallel primary agents balanced in latency
Timeline reconstruction	DeepSeek-V3.2	Novita / together via HF	Strong multi-hop/temporal reasoning across disparate evidence sources
Threat attribution (RAG)	Kimi K2.6	Together/Fireworks via HF	Best long-context handling for retrieving and reasoning over ATT&CK/CVE corpora
Memory forensics	Qwen3 32B	Nscale/Deepinfra via HF	Mid-size, good general performance, moderate volume of memory dump summaries
Identity and cloud	Prism-ML-Ternary-Bonsai-27B	Together AI via HF	Smaller specialized model sufficient for structured AD/cloud audit log analysis
Malware behavior & code stylometry	DeepSeek-R1-Distill-Qwen-14B	Nscale via HF	Strongest reasoning model for code-structure analysis (AST/PDG) and authorship inference
Insider threat	MiniMax-M2.5	Novita/Featherless AI via HF	Good at sentiment/behavioral narrative synthesis from communications data

Proponent (debate)	Nemotron-3 Super	Featherless AI via HF	Strong hypothesis construction, one tier below Ultra to keep debate asymmetric-but-capable
Critic (debate)	Llama3.3-70B-Instruct	Groq/Novita via HF	Independent model family from proponent, reduces correlated blind spots in the debate
Judge (debate)	Nemotron-3 Ultra	Together AI via HF	Same top-tier model as supervisor — final arbitration needs the strongest available reasoning
Guardrail Tier 3 semantic check	Qwen2.5-7B-Instruct	Novita/Together via HF	Fast, cheap, only invoked for containment/high-risk writes — small model is enough given Tier 1/2 already filtered
Report generation	Llama-3.3-8B-Instruct	Featherless AI via HF	Formatting/synthesis into a fixed template, deterministic task, doesn't need a large model

Model Agent Role(s)	Input	Output	What It Does
Nemotron-3 Ultra (Supervisor + Judge)	Supervisor: SIEM alert/investigator query, live blackboard state, agent observations. Judge: Proponent's hypothesis + Critic's counter-argument, both with DFKG citations	Supervisor: dispatch decisions, dead-end/timeout calls. Judge: accept/reject verdict, confidence delta score	Runs the two highest-stakes reasoning points in the system — orchestrating which agents fire and when (Supervisor), and arbitrating the adversarial debate to reach a final, defensible verdict (Judge)
Qwen2.5-7B-Instruct (Evidence Collection + Guardrail)	Tier 3 Collection: raw evidence source pointers, tool schemas. Guardrail: a proposed agent output/write action pending approval	Collection: fetched/normalized evidence artifacts. Guardrail: SAFE/UNSAFE semantic classification	Handles the two highest-volume, lowest-complexity jobs — fetching evidence via tool calls, and a fast final semantic check before any write/report action is allowed through
Qwen2.5-72B-Instruct (Log Analysis)	Normalized OCSF log batches (post-ingestion)	Flagged suspicious-activity findings, published to the DFKG	Detects anomalies and suspicious patterns across high-volume system/application logs
Llama3.3-70B-Instruct (Network Forensics + Critic)	Network: normalized Zeek/Suricata/PCAP events. Critic:	Network: identified C2/exfiltration/anomaly findings. Critic:	Analyzes network traffic for compromise indicators, and separately serves as the debate's dissenting voice — deliberately

	Proponent's hypothesis + DFKG read access	counter-argument with independent citations	a different model family from the Proponent to avoid shared blind spots
DeepSeek-V3.2 (Timeline Reconstruction)	Correlated evidence from the DFKG (all primary-tier findings)	Chronologically ordered attack timeline, written back as timeline edges	Synthesizes fragmented evidence into a single ordered attack narrative — the data source for both the report's Timeline section and the interactive dynamic timeline
Kimi K2.6 (Threat Attribution)	DFKG timeline + retrieved MITRE ATT&CK/CVE/NIST context (RAG)	Attack-pattern tags, threat-actor attribution candidates	Matches observed behavior to known techniques and threat actors, chosen specifically for long-context handling over the retrieved corpus
Qwen3 32B (Memory Forensics)	Volatility/memory-dump outputs, dispatched only on a primary-tier dead-end	Process injection/credential-dumping findings	Investigates memory artifacts when primary evidence alone doesn't resolve the case
Prism-ML-Bonsai-27B (Identity & Cloud)	AD/Kerberos/cloud audit logs, dispatched on dead-end	Compromised-identity and cloud-control-plane findings	Smaller specialist model for structured identity/cloud log analysis
DeepSeek-R2 (Malware & Stylometry)	Malware samples, sandbox telemetry, extracted code/scripts	Behavioral analysis + AI-authorship/stylometry classification	Detonates and analyzes malicious code, and determines whether recovered scripts were AI-generated or human-authored
MiniMax-M2.5 (Insider Threat)	Email/messaging bodies, DLP alerts, UEBA behavioral scores	Sentiment/behavioral-anomaly narrative	Scores non-signature insider-threat behavior across communications and activity baselines
Nemotron-3 Super (Proponent)	Full DFKG subgraph context	Attack hypothesis with mandatory DFKG citations	Constructs the case's leading theory as the debate's advocate — every claim must trace to a graph citation
Llama-3.3-8B-Instruct (Report Generation)	Judge's verdict, DFKG citations, blast-radius/attribution outputs	Populated 17-section court-ready report	Formats the investigation's conclusions into the fixed report structure — deterministic, low-reasoning task, so the smallest model in the roster

3. Dataset

3.1 Benchmark & Ground-Truth Datasets

Specula utilizes established public datasets for RAG grounding, knowledge lookup, and empirical evaluation. Crucially, no heavy LLM fine-tuning is performed—this avoids catastrophic forgetting and excessive compute overhead.

Dataset / Source	Purpose in Specula System	Evaluated Agent / Feature
CERT Insider Threat v6.2 (CMU SEI)	Grounding and evaluation for behavioral anomaly detection in enterprise logs.	Insider Threat Agent
CICIDS2017 / CICIDS2018	Network traffic evaluation containing labelled benign and multi-attack traffic (PCAPs & Zeek logs).	Network Forensics Agent
Digital Corpora Forensic Images	Disk images, memory dumps, MFT files, and user artifacts for timeline ground truth.	Memory Forensics Agent & Timestomping Feature
DARPA Transparent Computing	System provenance data with known manipulation events for temporal anomaly evaluation.	Timestomping Detection (Feature 3)
ForensicsData (ANY.RUN Triplets)	5,000+ Q-C-A triplets testing LLM-generated malware behavioral narrative accuracy.	Malware Behavior Agent
CFA-Bench + CyberSleuth	Network forensic reasoning benchmark measuring CVE detection and attack confirmation.	Network Forensics & Threat Attribution Agents
DFIR-Metric Benchmark	Multi-step reasoning evaluation measuring Task Understanding Score (TUS).	All Primary & Specialist Agents
AutoBnB-RAG Simulation Dataset	Agent strategy validation environment based on	Supervisor & Dynamic Attack Graph Agents

	Backdoors & Breaches tabletop framework.	
NVD / NIST CVE Database	Daily EPSS CSV dump and CVE vulnerability metadata for attack path weighting.	Dynamic Attack Graph Agent
MITRE ATT&CK STIX Dataset	Threat actor TTP profiles in machine-readable STIX format for attribution matching.	Threat Attribution Agent

3.2 Novel Synthetic Dataset: Code Stylometry Benchmark

To fill an identified research gap—the total absence of public datasets containing malware scripts labeled by whether they were generated by specific LLMs (GPT-4, Claude, Gemini) versus human authors—this project creates a novel benchmark dataset.

- **Data Collection:** Human samples are collected from MalwareBazaar and pre-2023 VirusTotal submissions (ensuring clean human ground truth). AI samples are generated by submitting identical attack specifications to GPT-4, Claude Sonnet, and Gemini Pro across Python, PowerShell, Bash, and JavaScript.
- **Dataset Scale:** 500 samples per class across four class labels: {GPT4, Claude, Gemini, Human} (2,000 samples minimum; 5,000 target for statistical reliability).
- **Labeling Methodology:** Human samples are verified via git blame timestamps and pre-LLM repository creation dates. AI samples are labelled by generating model. Inter-annotator agreement is validated via Cohen's Kappa score.
- **Preprocessing & Feature Extraction:** All samples undergo Unicode NFKC normalization to prevent homoglyph contamination, comment stripping to eliminate author metadata, Abstract Syntax Tree (AST) extraction (via ast, javalang, pycparser, powershell-ast, bashlex), and lexical/syntactic feature vectorization.

4. Deployment

4.1 Deployment Architecture & Platform

Specula is deployed as a containerized, locally hosted microservices stack designed for air-gapped Security Operations Center (SOC) environments where evidence cannot be transmitted to external cloud providers. An optional cloud-API mode is supported for non-air-gapped deployments.

- **Platform:** Local bare-metal server or on-premise virtual machine (Ubuntu 24.04 LTS primary, Windows Server 2022 secondary).

- **Minimum Hardware Spec:** 16GB RAM, 8-core CPU, 512GB SSD. Uses quantised local Qwen 7B Q4 via Ollama (No GPU required).
- **Recommended Hardware Spec:** 32GB RAM, 16-core CPU, 1TB NVMe, NVIDIA RTX 3060 12GB GPU for accelerated inference.
- **Orchestration:** Docker + Docker Compose for single-node SOC deployment; Kubernetes optional for multi-node enterprise SOC.

4.2 Microservices Architecture & Stack

Microservice Component	Technology Stack	Port / Interface & RAM Budget
Log Ingestion Agent	Filebeat + Go custom processor	Ships SHA-256 hashed logs to Kafka (~100MB RAM)
Message Broker	Apache Kafka (single broker)	Internal Queue :9092 (~300MB RAM)
Raw Evidence Store	Quickwit (append-only index)	REST API :7280 (200–400MB RAM)
Knowledge Graph	Neo4j Community + APOC plugin	Bolt :7687 / HTTP :7474 (512MB–1GB RAM)
Vector Index	FAISS IndexIVFPQ	Python in-process library (~200MB RAM)
Embedding Store	ChromaDB	HTTP :8000 (200–400MB RAM)
State & Debate Cache	Redis	Internal :6379 (100–200MB RAM)
Asset / CVE Store	DuckDB local store	In-process library (50–200MB RAM)
LLM Inference Engine	Too many to list(Quantised)	HTTP :11434 (4–6GB RAM)
Agent Orchestration	LangGraph (Python)	Internal state graphs (~200MB RAM)

MCP Data Gateways	FastAPI microservices	Internal :8100–8105 (~100MB RAM)
Blockchain Ledger	Hyperledger Fabric	Internal :7050 (~300MB RAM)
HITL Analyst Dashboard	FastAPI backend + React frontend	HTTPS :443 (~100MB RAM)

4.3 Deployment Workflow

- **Step 1 — Infrastructure Provisioning:** docker-compose up -d initializes all microservices. Neo4j, Quickwit, and Kafka start with append-only forensic configurations and write locks.
- **Step 2 — Evidence Ingestion:** Filebeat agents ship system logs. Network taps forward PCAPs. API connectors pull cloud audit trails.
- **Step 3 — OCSF Normalization:** FastMCP servers normalize incoming evidence to Open Cybersecurity Schema Framework prior to DFKG ingestion.
- **Step 4 — Autonomous Investigation:** SIEM alert triggers the Supervisor Agent. Tier 2 primary and Tier 3 specialist agents run in parallel via Blackboard pattern.
- **Step 5 — Adversarial QC & HITL Gate:** Adversarial ACH debate validates findings. Supervisor escalates blast radius and confidence scores to the React analyst dashboard.
- **Step 6 — Report Delivery:** Court-ready PDF is generated by ReportLab with appended case-level Merkle root and VCT audit proof.

5. Target Users

Target User Group	Operational Context	How Specula Benefits Them
Digital Forensic Investigators	Law enforcement, private forensic firms, and IR consulting practices.	Reduces evidence correlation from hours to 5 minutes; outputs court-ready reports with cryptographic provenance.
SOC Analysts (Tiers 1–3)	Enterprise SOC teams triaging alerts in real time.	Provides pre-correlated, ranked attack narratives rather than raw alert

		queues, dramatically reducing fatigue.
Incident Response Teams	Emergency IR teams responding to active network breaches.	Compresses timeline reconstruction from days to minutes and highlights unchained residual risks for hardening.
Legal & Compliance Teams	In-house legal counsel needing admissible evidence for court.	Provides cryptographically signed, tamper-evident VCT transcripts satisfying Daubert legal admissibility standards.
Academic Researchers	University research groups in AI security and DFIR.	Provides 12 novel features, a novel synthetic code stylometry benchmark, and an open evaluation framework.

6. Input to the Application

6.1 Accepted Evidence Input Sources (14 Categories)

Input Category	Specific Technical Sources	Consuming Agent(s)
System Logs	Windows EVTX, Linux auditd syslogs, Sysmon logs	Log Analysis & Evidence Correlation Agents
NTFS Artifacts	\$MFT, \$USNjrnI, \$FILE_NAME & \$STANDARD_INFORMATION	Memory Forensics & Timestomping Module
Network Logs & PCAPs	Suricata IDS, Zeek connection/DNS/HTTP logs, raw PCAP	Network Forensics Agent
AD & Cloud Audit	Active Directory LDAP/Kerberos, AWS CloudTrail, Azure Logs	Identity & Cloud/Container Agents

EDR & UEBA Telemetry	CrowdStrike Falcon, SentinelOne, Wazuh, Splunk UBA	Memory Forensics & Insider Threat Agents
Malware Samples	YARA-matched binaries, PE headers, CAPE / ANY.RUN JSON	Malware Behavior & Code Stylometry Agents
Email & Messaging	Raw EML/MBOX files, Slack API, Teams Graph API, SharePoint	Evidence Correlation & Insider Threat Agents
Memory Dumps	Volatility 3 outputs, WinPmem/LiME memory images	Memory Forensics Agent
Browser Artifacts	Chrome/Firefox/Edge SQLite databases (History, Cookies)	Evidence Correlation Agent
Threat Intel Feeds	MITRE ATT&CK STIX, MISP IOC feeds, VirusTotal, NVD CVE	Threat Attribution & Malware Behavior Agents
Container / K8s Logs	Kubernetes audit logs, Falco runtime security, Docker logs	Cloud & Container Agent
Vulnerability Scans	OpenVAS / Nessus / Qualys API outputs, software manifests	Dynamic Attack Graph Agent
Cloud Topology	AWS/Azure/GCP Cloud Logging SDK, Cartography asset graphs	Cloud & Container Agent
Investigator Query	Natural language query submitted via React dashboard	Supervisor Agent (routes to specialists)

6.2 Input Capture, Processing, and Validation Workflow

- **Capture Channels:** Automated pulling via Filebeat agents and API connectors (boto3, falconpy); manual upload of disk images/malware binaries via dashboard; on-demand natural language query submission.

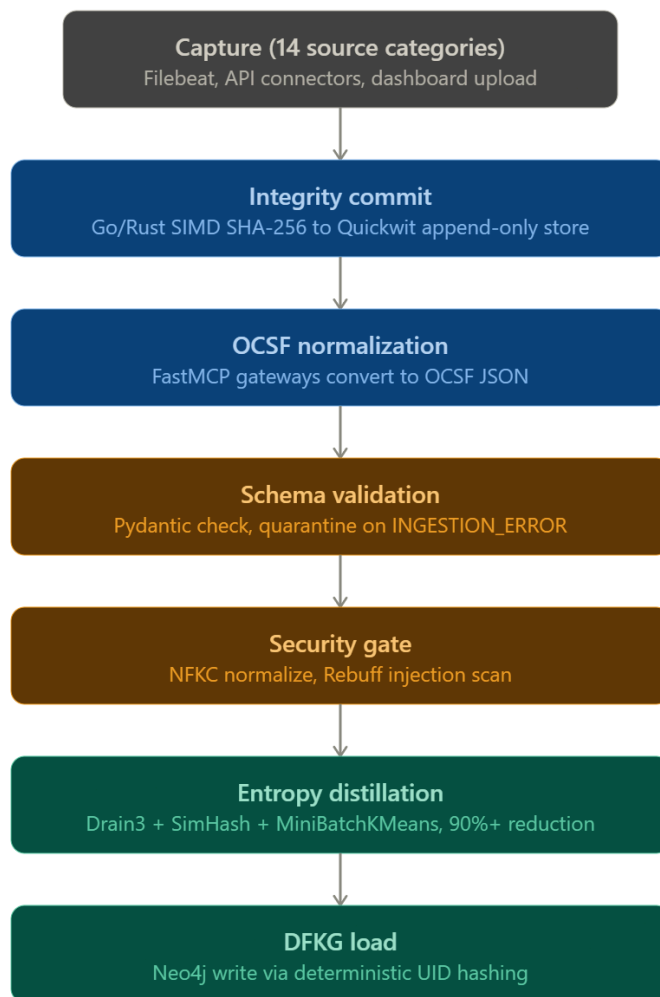
- **OCSF Normalization:** All raw evidence is converted to Open Cybersecurity Schema Framework (OCSF) standard JSON via FastMCP gateways before touching agent prompts.
- **Schema Check:** Pydantic models validate all OCSF events. Violations raise INGESTION_ERROR and quarantine malformed records.
- **Integrity Verification:** Go SIMD processor computes SHA-256 hash of raw logs upon arrival, committing them to Quickwit append-only store. Any mismatch raises TAMPER_DETECTED.
- **Security Gate:** All evidence text passes through Unicode NFKC normalization and Rebuff prompt injection detection before agent consumption.

Priority Tier	Rank	Log / Evidence Type	Primary Consuming Agent / Module	Core Forensic Purpose & Dependency
Tier 1: Critical (Core Pillars)	1	System Logs (EVTX, Linux syslogs, Sysmon)	Log Analysis & Evidence Correlation	Foundational telemetry; feeds initial timeline, entropy distillation, and primary DFKG nodes. Google Docs+ 2
	2	NTFS Artifacts (\$MFT,\$USNjrnI, SI/FN)	Memory Forensics & Timestomping Module	Crucial for anti-forensic timestomping detection via XGBoost before timeline synthesis. Google Docs+ 1
	3	Active Directory & Auth Logs (LDAP, Kerberos, MFA)	Identity & Cloud Agents	Tracks identity context, privileges, and initial compromise vectors across hosts. Google Docs+ 3
	4	EDR Telemetry & Memory Dumps (CrowdStrike, Volatility)	Memory Forensics Agent	Deep endpoint process execution tracing, volatile memory analysis, and root cause mapping. DOCX+ 1
Tier 2: High (Network & Cloud Context)	5	Network Logs & PCAPs (Suricata, Zeek, Raw PCAP)	Network Forensics Agent	Maps network-level lateral movement, C2 channels, and calculates exfiltration volumes. Google Docs+ 2
	6	Cloud Audit & Container Logs (CloudTrail, K8s, Falco)	Cloud & Container Agent	Expands investigation into cloud control planes, multi-cloud infrastructure, and container runtime events. DOCX+ 1
	7	Malware Samples &	Malware Behavior & Code Stylometry	Tiered sandbox execution (YARA → Speakeasy → CAPE/DRAKVUF) and LLM

		Binaries (YARA, PE Headers, CAPE)		code stylometry analysis. Google Docs+ 3
Tier 3: Medium (Specialized Threat Intelligence)	8	UEBA Data & Messaging Logs (DLP, Slack, Teams, EML)	Insider Threat Agent	Mandatory inputs for the LSTM Autoencoder and spaCy NER to score non-signature behavioral anomalies and exfiltration. Google Docs+ 1
	9	Threat Intel Feeds (MITRE ATT&CK STIX, NVD)	Threat Attribution Agent	Required for Smith-Waterman sequence-aligned threat actor attribution and TTP matching. Google Docs+ 2
	10	Vulnerability Scans (OpenVAS, Nessus, Qualys)	Dynamic Attack Graph Agent	Feeds EPSS/CVSS scores into Dijkstra's negative log network graph for residual risk mapping. Google Docs+ 3
Tier 4: Supporting (Enrichment)	11	Browser Artifacts (Chrome/Firefox SQLite)	Evidence Correlation Agent	Provides web activity, download histories, and phishing landing page entry points. DOCX
	12	Cloud Topology & Asset Data (Cartography)	Cloud & Container Agent	Enriches DFKG host nodes with network mapping and asset sensitivity classifications. Google Docs+ 1

Ingestion pipeline

The pipeline turns 14 heterogeneous evidence sources into OCSF-normalized, tamper-evident, DFKG-ready records before any agent touches them.



Python

```
# Stage 1-2: capture + integrity commit
async def ingest_source(raw_bytes: bytes, source_type: str) -> str:
    doc_hash = sha256_simd(raw_bytes) # Go/Rust SIMD hasher, called
    via FFI
    await quickwit_client.commit(index="raw-evidence",
    doc=raw_bytes, id=doc_hash)
    await vct_chain.append(doc_hash) # hash chain, atomic level
    return doc_hash
```

```

# Stage 3: OCSF normalization (per-source FastMCP gateway)
def normalize_to_ocsf(raw_bytes: bytes, source_type: str) -> dict:
    parser = OCSF_PARSERS[source_type] # e.g. evtx_parser,
zeek_parser, cloudtrail_parser
    return parser.to_ocsf(raw_bytes)

# Stage 4: schema validation
def validate(event: dict) -> OCSFEvent | None:
    try:
        return OCSFEvent.model_validate(event) # Pydantic
    except ValidationError:
        quarantine_store.put(event) # raises INGESTION_ERROR
downstream
        return None

# Stage 5: security gate
def security_gate(event: OCSFEvent) -> OCSFEvent:
    event.text_fields = unicodedata.normalize("NFKC",
event.text_fields)
    if rebuff.detect_injection(event.text_fields).is_injection:
        event.flag = "QUARANTINED_INJECTION"
    return event

# Stage 6: entropy distillation (batched, not per-event)
def distill_batch(events: list[OCSFEvent]) -> list[OCSFEvent]:
    templates = drain3.mine(events)
    templates = simhash_dedupe(templates) # anti template-poisoning
    clusters = MiniBatchKMeans(n_clusters=k).fit(templates.vectors)
    return keep_high_entropy(events, clusters) # low-entropy →
summarized, anomalies kept whole

# Stage 7: DFKG load
def load_to_dfkg(event: OCSFEvent):
    uid =
sha256(f"{event.device_id}:{event.path}:{event.db}:{event.row}".enc
ode())

```

```
neo4j_session.run(
    "MERGE (n:Entity {uid: $uid}) SET n += $props",
    uid=uid.hexdigest(),
    props=event.model_dump()
)
```

Kafka sits between stages 2 and 3 as the durability buffer (Filebeat → Kafka → FastMCP consumer group), so a normalization crash never loses raw evidence — replay from the Kafka offset, not from source.

8. Agentic Architecture — ReAct Agents, Data Flow, and Feedback Loops

Every agent in Specula is a ReAct agent: each runs its own internal Thought → Action → Observation loop, calling tools (DFKG queries, RAG retrieval, sandbox detonation) and revising its reasoning until it reaches a terminal answer or hits a dispatcher-defined dead-end/timeout condition. The architecture is represented as a directed graph, not a hierarchical tree, because agent communication is not purely top-down: agents write to and read from the shared Digital Forensic Knowledge Graph, specialists escalate observations back to the Supervisor, and the debate tier runs an explicit multi-round revision loop. Four loops are explicit and load-bearing:

- **Debate revision loop:** Judge → Proponent, up to 3 rounds, terminating when the confidence delta between Proponent and Critic falls below 0.05.
- **Guardrail escalation chain:** Tier 1 (regex/AST, <5ms) → Tier 2 (MiniLM classifier, <50ms) → Tier 3 (LLM semantic check, 1–3s); any tier failing routes directly to the HITL dashboard rather than continuing down the chain.
- **HITL feedback loop:** an analyst's APPROVE / REJECT / CLARIFY decision returns to the Supervisor, which can re-dispatch to any tier — the only edge that re-enters the top of the graph after initial dispatch.
- **Specialist dead-end conditional dispatch:** Memory Forensics, Identity & Cloud, Malware & Stylometry, and Insider Threat agents are only invoked when the primary tier's evidence reaches a dead-end, not unconditionally on every case (a direct implementation of the "selective agent pruning" cost-control requirement raised during project brainstorming).

Solid arrows in the architecture diagrams represent dispatch/control or a primary write into the DFKG; dashed arrows represent reads or bidirectional queries. Every edge is explicitly labeled with the data it carries (e.g. "write timeline edges," not just an unlabeled arrow), so the graph is legible to an implementer without narration.

9. Specula Runtime Harness

The runtime harness is the infrastructure every deployed agent depends on to execute safely and predictably — distinct from the Antigravity IDE development harness used to build Specula, which is out of scope here.

9.1 Event & Data Infrastructure

- **Message broker (Kafka):** the backbone of the Blackboard architecture. Agents do not pass large JSON payloads directly to each other; they publish findings to topics (e.g. `logs.normalized.ocsf`, `findings.lateral_movement`), with a schema registry enforcing versioned OCSF contracts on every topic and dead-letter topics catching any message a consumer cannot process.
- **Neo4j + APOC triggers:** the DFKG is the shared memory layer. Triggers watch for milestone graph-write patterns (e.g. `NTFS_TIMESTOMPING_DETECTED`) and emit notifications back to the message broker — registered with a 5-second async debounce window so a burst of simultaneous writes (e.g. 200 hosts encrypting at once) does not cause an agent wake-storm.
- **Cryptographic ledger:** an append-only WORM datastore anchoring Merkle roots for the Forensic Preservation Layer.

9.2 MCP Server Layer

- **mcp-dfkg-cypher:** translates an agent's semantic request into deterministic, parameterized Cypher queries, and centralizes the typed supernode check (`apoc.node.degree(n, rel_spec)`, always directional and typed, never an untyped or pattern-comprehension-based degree check).
- **mcp-vmi-sandbox:** secure bridge to the DRAKVUF/CAPE detonation environment.
- **mcp-threat-intel:** unified connector across VirusTotal, AlienVault OTX, and MITRE ATT&CK RAG profiles, with circuit breakers and provider fallback owned centrally rather than reimplemented per agent.
- **mcp-vct-ledger:** exposes Merkle/hash-chain writes and citation lookups through the same tool-call interface pattern as a DFKG query.
- **mcp-dataset-eval:** exposes CICIDS/CERT/DFIR-Metric ground truth as a runtime-queryable tool, not just an offline benchmark.

9.3 Agent Skills Library

Every skill is a versioned unit with a manifest declaring which agent role(s) may invoke it, its MCP dependencies, and its cost/latency class — enforcing least-privilege scoping (the Report Generation agent, for example, has no access to `mcp-vmi-sandbox`). Skills include `OCSF_Normalizer_Skill`, `Temporal_Drift_Calculator` (timestomping detection), `Calculate_Blast_Radius`, `Entropy_Distillation_Skill`, `Code_AST_Extraction_Skill`, `Smith_Waterman_Attribution_Skill`, and `Dijkstra_Attack_Graph_Skill`. Deterministic skills (blast radius, Cypher translation) carry checked-in golden-fixture unit tests, since a silently wrong

deterministic skill corrupts the shared graph for every downstream agent without anyone noticing until the final report is wrong.

9.4 Operational Rule Engine

Rules are scoped per agent role rather than injected as one monolithic file, are hot-reloadable from a rules service rather than baked into container images, and include an explicit degradation policy (what an agent does when a tool times out mid-loop: escalate to HITL rather than hallucinate a plausible answer).

9.5 ReAct Loop Runtime

- **Persistent checkpointing:** (Postgres/Redis-backed) so an agent's loop state survives a container restart mid-investigation.
- **Loop budget enforcement:** maximum iterations and tool calls per agent per task, with graceful timeout returning a partial observation to the Supervisor.
- **Context-compaction skill:** auto-invoked as an agent's transcript approaches its model's context budget.
- **Scratchpad/DFKG separation:** in-flight, unconfirmed reasoning lives in Redis working memory; only confirmed findings are promoted to the DFKG, so speculative intermediate thoughts never pollute the shared graph.

9.6 Observability, Model Routing, and Security Hardening

A central model router owns provider fallback across the twelve-plus LLM APIs in use; a fallback event on a safety-critical agent (Judge, Guardrail Tier 3) flags that case for HITL review, since the confidence assumptions behind the project's accuracy targets no longer hold under a substituted model. Distributed tracing (case_id/trace_id propagated through Kafka headers and DFKG node properties) makes every finding traceable end-to-end. Skills that parse attacker-controlled input run in sandboxed/restricted containers, and every MCP tool call — not just agent conclusions — is logged into the same VCT Merkle chain, since Daubert admissibility requires showing why an agent examined a piece of evidence, not only what it concluded.

10. Finalized Evidence Ingestion Pipeline

The rough single-file sketch shown earlier in this document (Section 6, ingest_source/normalize_to_ocsf/validate/security_gate/distill_batch/load_to_dfkg) was an early draft. It has since gone through five rounds of architectural review and is superseded by a fully specified, review-hardened implementation plan. The corrected pipeline order and its key differences from the original sketch:

Capture Channels -> Integrity Verification (SHA-256 SIMD -> Quickwit append-only store, VCT atomic hash chain) -> Security Gate, source-type branched: text-native sources: NFKC + zero-width strip + Rebuff, directly binary sources (EVTX, \$MFT/\$USNjrnI, PCAP): structural field extraction FIRST, then per-field sanitization -- NFKC/Rebuff cannot run on undifferentiated binary bytes -> OCSF Normalization & dual timestamp preservation (raw_source_timestamp

AND utc_timestamp both retained permanently, never overwritten; clock_skew_offset_ms recorded; NTFS timestamping fields preserved for F17) -> Wire Serialization & Schema Registry (Confluent JsonSerializer / JSONDeserializer, symmetric on producer and consumer -- enforcement at the wire level, not just in application code) -> Event Broker: Kafka, partitioned strictly by hash(canonical_host_id) -- never by case_id, so case reassignment never breaks per-host chronological ordering; case_id tagged at produce time via a persistent ActiveCasesCache (a Redis key-value store, NOT PubSub-as-store -- a restarted producer must resolve the correct active case with zero reliance on a missed notification); dead-letter topic for any post-validation processing failure -> Analytical Abstraction: Drain3 template mining + SimHash anti-poisoning + MiniBatchKMeans entropy clustering, collapsing low-entropy events while preserving high-entropy anomalies intact; Redis offset checkpointing before manual Kafka commit, with a degraded fallback path (native commit + reconciliation queue) if Redis is unreachable -> Knowledge Graph Ingestion: parameterized Cypher MERGE only (never string-concatenated, even from a hardcoded label list); debounced APOC triggers (5000ms) for milestone pattern detection ONLY -- dead-end detection is a Supervisor-side inactivity heuristic, not something a database trigger can observe; supernode pruning evaluated at query time via apoc.node.degree(n, rel_spec) with an explicit, typed, directional relationship spec, never an untyped or pattern-comprehension-based degree check; historical case-tagging backfill runs once per case-open event over indexed composite (host, timestamp) fields across all relevant labels including NetworkEndpoint -> Vector Indexing: ChromaDB case_evidence_embeddings, kept explicitly separate from the FAISS threat-intel corpus index used by Threat Attribution.

Cross-source entity resolution (CanonicalEntityResolver) maps heterogeneous identifiers — hostname, IP, cloud device ID — to one canonical UID before hashing, with IP-to-host mappings time-bounded against DHCP lease validity windows so a dynamic IP reassignment does not silently merge two different hosts into one entity. Phase 1 covers five source types in full (System Logs, NTFS Artifacts, Network Logs, AD/Auth Logs, Cloud Audit); the remaining nine of the fourteen input categories are explicitly deferred to Phase 2/3 rather than implemented partially. Compression is verified against serialized byte volume, not event count, since a verbose summary record can pass a count-based reduction test while barely reducing the actual data volume fed to the LLM — the actual quantity the project's 90% compression target is defined against. A complete, unambiguous implementation specification — including exact file paths, function signatures, and a full catalogue of mistakes to explicitly avoid at each pipeline stage — is maintained as a standalone engineering document (specula_ingestion_final_plan.md) alongside a corresponding pytest suite under tests/ingestion/, and is treated as authoritative over this summary.

11. Report Generation Format

Specula's court-ready report follows a fixed 17-section structure, with every section traceable to a specific agent or DFKG artifact rather than freely generated:

Report Section	Primary Data Source
Executive Summary	Judge's final ACH verdict + blast radius score

Incident Overview	DFKG root incident node + earliest confirmed timeline entry
Scope and Objectives	Supervisor's case initialization parameters
Investigation Methodology	Active rule-engine ruleset + skill manifest used
Evidence Inventory	Evidence Collection Agent + Quickwit raw evidence index
Chain of Custody	VCT Merkle chain (atomic, session, and case level)
Acquisition Process	Evidence Collection Agent's capture logs + hash commit records
Hash Values	SIMD SHA-256 outputs at ingestion, per artifact
Timeline of Events	Timeline Reconstruction Agent output — same data feeds the dynamic attack timeline (Section 13)
Technical Findings	Log Analysis + Network Forensics + specialist tier observations, DFKG-cited
Indicators of Compromise	Threat Attribution Agent + mcp-threat-intel results
Malware / Artifact Analysis	Malware & Stylometry Agent, tiered sandbox output
Impact Assessment	Calculate_Blast_Radius skill output
Applicable Laws	Rule engine's jurisdiction-specific legal reference table
Conclusion	Judge Agent's terminal verdict
Recommendations	Report Generation Agent synthesis from the Dynamic Attack Graph's residual-risk list
Appendices	Raw DFKG subgraph export + full VCT audit transcript

12. Dynamic Attack Timeline

Alongside the static PDF report, Specula generates an interactive, graph-type attack timeline — a horizontal, stage-by-stage rendering of the reconstructed attack chain, grounded directly in DFKG node citations. Each stage is expandable to reveal its underlying evidence (source agent, DFKG node UUIDs, severity), with a severity filter and export-to-image function for embedding a static snapshot into the PDF's Timeline of Events section. This is deliberately a separate, explorable artifact rather than a diagram baked into the report — it is the view a HITL reviewer or SOC analyst would actually interact with, distinct from the fixed narrative the PDF presents to legal/executive stakeholders.

13. Expected Outcome

13.1 Primary System Output Deliverables

- **Deliverable 1 — Unified Chronological Attack Supertimeline:** Cross-correlated sequence of confirmed attack events exported to Timesketch for interactive review.
- **Deliverable 2 — Executive Summary Narrative:** Plain-English explanation of attack mechanics generated for non-technical stakeholders, with every claim citing DFKG graph nodes.
- **Deliverable 3 — MITRE ATT&CK Mapping & Threat Actor Attribution:** Smith-Waterman sequence alignment + Jaccard similarity top-3 threat actor candidates with explicit confidence scores.
- **Deliverable 4 — DFKG Subgraph Visualizations:** Neo4j interactive relationship network rendering full attack chains (User -> Email -> Binary -> C2).
- **Deliverable 5 — Dynamic Attack Graph & Remediation Priority List:** Exploitation path map isolating unchained vulnerabilities with EPSS > 0.3 as residual risk priorities.
- **Deliverable 6 — Court-Ready PDF Investigation Report:** ReportLab PDF featuring Executive Summary, ATT&CK Heatmaps, Blast Radius Scores, and Cryptographic Appendix.
- **Deliverable 7 — Cryptographic VCT Audit Transcript:** Three-tier Merkle proof chain anchored to Hyperledger Fabric for legal admissibility under Daubert standards.

13.2 Quantitative Performance Targets

Performance Metric	Specula Target	Traditional SIEM / Single-LLM Baseline
Evidence Correlation Accuracy	95%	72%
Attack Timeline Reconstruction Accuracy	96%	68%
Threat Attribution Accuracy	94%	70%
Total Investigation Duration	5 minutes	45 minutes
Investigation Time Reduction	89% reduction	Baseline
AI Hallucination Rate	< 5%	18% (single-LLM baseline)

Information Retention Rate (Compression)	>= 95% entity retention at >=90% compression	N/A (No compression)
--	--	----------------------

13.3 Real-World Use Cases & Practical Impact

- **Use Case 1 — Enterprise Ransomware Incident:** A manufacturing firm detects file encryption across 200 hosts. Specula ingests EVTX, Sysmon, and PCAPs, outputting the initial access vector, lateral movement chain, C2 servers, blast radius (47 affected hosts), and remediation steps in 5 minutes (vs. 6-hour manual response).
- **Use Case 2 — Insider Threat Data Exfiltration:** A financial institution suspects data theft. The Insider Threat Agent analyzes email bodies, DLP alerts, and USB logs. The LSTM autoencoder detects behavioral deviation 3 weeks prior, outputting a court-ready narrative with signed VCT Merkle proofs.
- **Use Case 3 — CI/CD Supply Chain Attack:** Cloud & Container Agent analyzes K8s pod logs. Code Stylometry Agent determines injected build script exhibits AI-generated characteristics. Dynamic Attack Graph maps exploitation paths, completing cross-layer investigation in 5 minutes.

14. Known Architectural Gaps, Cross-Document Conflicts, and Recommended Mitigations

14.1 Cross-document conflict — flag before anything else

- The documents *specula_architecture_v3_feature_map.html* and *specula_runtime_harness.md* disagree with this Master doc on two load-bearing mechanics; this Master doc (Section 10, "five rounds of architectural review") is authoritative:
 1. **Specialist DFKG writes.** Older docs suggest specialists write directly to DFKG via harness, bypassing Kafka. Per Section 9.1 of this document, *all* agents publish findings to Kafka topics with no direct-write path.
 2. **Dead-end detection mechanism.** Older docs specify an APOC graph trigger. Per Section 10 of this document, dead-end detection is an explicit **Supervisor-side inactivity heuristic**. APOC triggers are scoped only to milestone pattern detection (e.g. NTFS_TIMESTOMPING_DETECTED), because dead-end detection is not something a database trigger can observe.
- **Recommendation:** Mark the HTML feature map and runtime harness doc as superseded/historical, or revise them to match, so no implementer works from the older mechanic by mistake.

14.2 Issues from earlier review that this Master doc already resolves — document them as resolved, with citation

Issue	Where it was raised	How this doc resolves it	Section citation
Which logs feed each Stage 1 agent	Review	14-category input table & 12-row priority-tier table	6.1, 6.2
Retry idempotency / duplicate writes	Review	Deterministic UID hashing + parameterized Cypher MERGE	9.3, 10
Prompt injection via evidence content	Review	Mandatory Security Gate (NFKC + Rebuff)	6.2, 10
Critic "winning" auto-accepted	Review	Judge is sole arbiter using ACH matrix	2.4 (feature 4)
Debounced APOC triggers merging dead-ends	Review	APOC triggers no longer handle dead-ends	14.1
Code-parsing parser-attack-surface risk	Review	Sandboxed/restricted containers for skill execution	9.6
Debate stage handed full subgraph context	Review	APOC-bounded BFS retrieval limits	2.4 (feature 4)
Audit-trail asymmetry between tiers	Review	All writes route through mcp-dfkg-cypher to VCT ledger	9.2, 9.6

14.3 Issues raised earlier that remain open — add with proposed solutions

Open issue	Why it's still open	Proposed solution
Debate round-cap with no convergence	No stated outcome for 3 rounds	Force-escalate to HITL with final positions
Checkpointing backing store ambiguity	Postgres/Redis roles unclear	State Postgres as system of record, Redis as cache
Rule-engine versioning mid-case	No case-level pinning	Write rules_version property on case open

Replay harness determinism	Not addressed	Snapshot RAG corpus/responses keyed to trace_id
Fallback-confidence-flagging scope	Only Judge/Guardrail covered	Extend to Threat Attribution and primary-tier findings
Golden-fixture coverage too narrow	Only 2 skills named	Extend to all 5 deterministic skills
Threat-intel provider fallback invisible	Provider fallback transparent	Apply same flagging pattern as model fallback
Report-generation consistency gate	No "case frozen" signal	Add case-state transition (frozen) gated on zero lag
Timeline/attribution staleness	No explicit waiting for specialists	Explicitly define dependency or re-run trigger
No bound on HITL re-dispatch	No attempt cap	Add attempt counter; force manual override after N
Tracing gap at HITL	Dashboard not in trace	Propagate trace_id into dashboard payload
No feedback loop from overrides	No consumption of agreement rate	Trigger observability review on low agreement

14.4 New issues found specifically in this Master doc

- 1. Kafka partitioning by *hash(canonical_host_id)* sacrifices cross-host ordering.**
Solution: state explicitly that all cross-host ordering must be resolved downstream from the *utc_timestamp* property at the DFKG layer, never inferred from Kafka consumption order.
- 2. Redis' blast radius as a single point of failure has grown.** Solution: define an explicit degraded-fallback path for *ActiveCasesCache* lookups matching the pattern for the offset-checkpoint.
- 3. CanonicalEntityResolver's DHCP-lease-bounded IP resolution doesn't cover Container/K8s/Cloud logs.** Solution: extend *CanonicalEntityResolver* with a parallel cloud/container-native resolution path using cloud metadata IDs.
- 4. Phase-1 scope isn't reflected in the objectives/deliverables.** Solution: add a Phase 1/2/3 column to the Section 6.1 input-category table.
- 5. Guardrail Tier 3's model size is in tension with its role.** Solution: either justify why a 7B-class model is adequate for containment-gate reasoning, or reconsider the model assignment.

6. **Supervisor and Judge share a single model with no cross-check.** Solution: document why this asymmetry is acceptable or apply a similar diversity safeguard.
7. **Compression-target measurement point is underspecified.** Solution: pin the measurement point to raw OCSF payload bytes, excluding wire-format envelope overhead.

— *End of Proposal Document* —